南通理工學院

课程总结报告

课程名称　　软件专业综合实践

实训项目名称　　LIMS 实验室信息管理系统设计与实现

班级　软件 2301　　姓名　孙熙隆　　学号　2310501102

实训日期　2026.8.31-10.31　实训地点　数智楼-804

LIMS 实验室信息管理系统设计与实现

一、实训项目介绍

本实训围绕实验室信息管理系统（LIMS）开展，目标是打通检测机构从样品受理到报告交付的核心业务。系统以样品为主线，覆盖样品登记、分解排样、任务安排、检验数据录入、审核签发、报告下载和数据导出等环节。开发过程采用 VibeCoding 方式，通过 AGENTS.md 固定技术栈、目录结构、分层职责、安全约束与构建门禁，把自然语言需求逐步落实为可运行、可检查的功能。

项目最终完成服务信息、任务管理、样品管理、分解排样、项目检验、审核签发和基础数据 7 个业务模块，形成 26 个前端页面组件、14 个后端控制器和 12 个业务服务。业务上实现了“登记—分解—安排—检验—签发—报告/导出”的完整闭环，并针对退登记、退分解、退安排、转待检验、转待分解等场景提供反向流转能力。系统还提供项目与检验项目查询、文件导入导出、检验员匹配、报告模板选择及公共详情弹窗等支撑功能。

二、技术框架与系统设计

系统采用前后端分离架构。前端使用 Vue 3、TypeScript、Vite 和 Element Plus 构建单页应用，由 Vue Router 管理路由，Pinia 维护登录用户与权限状态，Axios 统一封装请求、令牌携带和异常处理。后端使用 Java 17、Spring Boot 3 和 MyBatis-Plus，按 Controller、Service、Mapper 分层组织代码，由统一响应对象返回结果，并通过全局异常处理将业务异常、参数错误和系统异常转换为一致的数据结构。MySQL 8 负责业务数据持久化，JWT 用于身份认证，Apache POI 负责报告与导出文件的实时生成。

数据库设计以 project 样品主表、prj_detail 检验项目明细、task 检验任务、lib 标准库、product 项目库以及 user、customer 等基础表为核心，通过项目编号、任务编号和人员编号建立关联。每个业务环节以状态字段驱动，前端根据状态决定可执行操作，后端再次校验状态和权限，避免仅依赖界面隐藏造成越权。接口层统一提供分页、模糊查询、区间筛选和排序能力，并对可查询实体与列设置白名单，既减少重复代码，也防止任意字段查询带来的安全风险。

在关键流程上，样品登记后进入分解，系统依据标准库生成检验项目明细；分解完成后按检验项目、检验员资质及样品类别方法码匹配任务；检验员录入结果后进入审核签发，签发完成即可生成并下载 CMA 或 CATL 报告。前后端通过开发代理联调，后端地址、数据库账号等敏感配置由环境变量或本地配置提供，不写入业务代码。

三、主要功能与运行效果

七个模块分别承担不同职责：服务信息模块集中管理项目库、在检项目和历史项目资料；任务管理模块提供监督任务管理及执行、抽样查询；样品管理模块支持采样单导入、登记确认、登记查询、数据导出与报告下载；分解排样模块支持单样品和批量样品的分解、任务安排；项目检验模块提供待检任务查询和检验数据录入；审核签发模块完成报告签发与检验汇总；基础数据模块维护用户、方法检验员、岗

位、标准库和项目库，并提供资料上传入口。

系统已在本地完成前后端联调，使用真实数据库数据验证登录、列表查询、详情查看、任务状态流转、报告下载及数据导出等关键链路。图 1 展示了在检项目查询页面，可按项目编号、名称、委托单位和阶段等条件筛选，并通过分页、详情弹窗查看完整业务信息；图 2 展示了报告下载页面，可依据样品信息和报告类型生成、下载对应文件。页面在加载、空数据、校验失败和业务异常时均给出明确提示，便于操作人员定位问题。

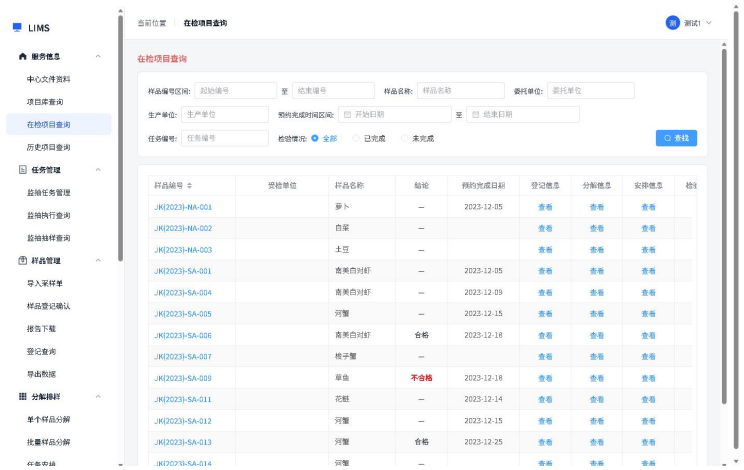


图 1 在检项目查询与筛选

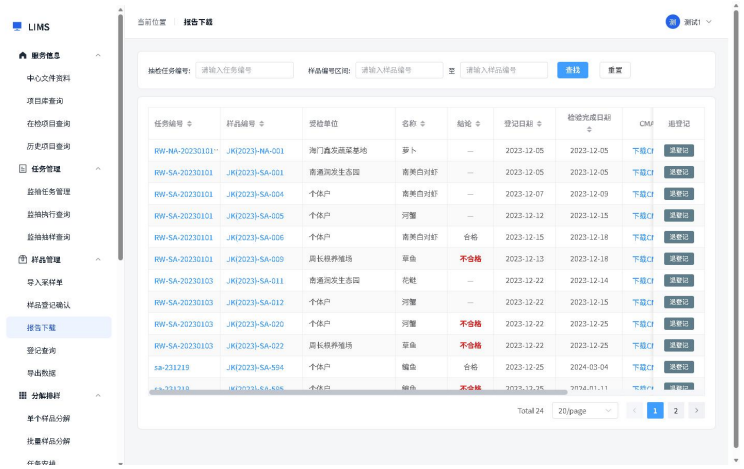


图 2 报告下载与结果反馈

构建与验证方面，后端可正常编译和启动，前端通过 TypeScript 类型检查、Vite 生产构建和 ESLint 检查；联调时不仅查看页面是否报错，还核对接口返回条数、关键字段和状态分布。涉及修改数据的测试先记录原值，验证后逐项还原，保证实训验证不破坏原有业务数据。

四、项目创新点

第一，采用可逆的状态机组织业务流转。系统将登记、分解、安排、检验和签发拆分为清晰阶段，在正向推进之外保留按阶段退回的能力，并通过状态判断和重复操作保护维持数据一致性。第二，将通用数据接口与业务接口结合，通过实体白

名单、列白名单统一实现分页和筛选，在控制代码重复量的同时兼顾查询安全。

第三，抽取编号详情、项目明细、文件下载、标准库导入等公共组件，把跨页面重复交互收敛为同一份实现，降低页面行为不一致的风险。第四，将 AI 辅助开发纳入工程约束：先把技术栈、权限红线、数据库规范和构建命令写入项目规范，再按“确认数据口径—实现接口—完成页面—构建回归—真实数据验证”的闭环推进。这种方式提高了页面和接口的实现效率，也让 AI 产出始终受到可编译、可检查和可回溯的工程门槛约束。

五、主要心得体会

这次实训让我认识到，软件项目的难点往往不在写出一个页面，而在准确理解业务和数据口径。旧库部分状态字段缺少说明，备注也可能为空，不能仅凭字段名称猜测含义。我通过按状态分组统计、对照样例数据和观察业务流程，逐步确认各字段的作用，并把无法完全证实的假设写进代码注释和汇报材料。这个过程让我明白，技术实现必须建立在对业务的诚实理解之上。

其次是验证闭环的重要性。代码能够编译、页面能够打开，并不代表功能一定正确。只有启动完整系统，使用真实数据走通查询、流转和文件生成，再核对返回条数与数据库状态，才能确认改动可靠。测试若写入业务数据，还要提前记录、事后还原；这既是对原系统负责，也是工程实践中应有的数据安全意识。

再次是规范与复用的价值。把分层职责、后端鉴权、敏感信息管理和构建门禁写进 AGENTS.md 后，AI 生成的代码更容易保持统一的风格和边界；把编号弹窗、详情表格、文件下载等重复逻辑抽成公共组件后，页面维护成本明显下降。遇到依赖受限、中文编码、基础数据缺失等问题时，也需要先定位约束，再选择改动最小、可以验证的方案，而不能只追求“看起来已经完成”。

最后是边界意识。AI 可以快速完成代码检索、页面搭建、接口编写和问题定位，但业务语义、验收标准和风险判断仍要由人确认。人需要把大目标拆成能够验证的小步骤，对每次产出检查证据，并对最终交付负责。通过本次实训，我完整经历了 LIMS 从需求梳理到联调交付的过程，对前后端协作、状态一致性、安全校验和 AI 辅助开发都有了更具体的理解，也提升了独立分析问题和持续推进任务的能力。